# Python Advanced Nuances

## Pre-Exam Tutorial Session

A Sleep-Deprived 2nd Year (+ Claude)

CSE Department

April 24, 2026

# Today's Topics


1. Built-in Methods Cheat Sheet
2. Python Core Nuances
3. Easy Problems
4. NumPy
5. Matplotlib

# Important Disclaimer

**Before We Start**

I am a **student**, not a professor. Manage expectations.

- **We don't judge here.** This apply to me also, please don't judge me by this TSC.
- **Nothing I say can be cited in court.** I accept zero liability. If I say something wrong, you can't use it to crib(or whatever you want to say that).
- **Slides made by Claude.** I also have endsems!!! Content verified by me(once) - mistakes: blame the AI (and a little bit me).

# String Methods

## Searching & Testing

```
s = "Hello, World!"
s.find('o')          # 4  (-1 if missing)
s.index('o')         # 4  (raises if missing)
s.count('l')         # 3
s.startswith('He')   # True
s.endswith('!')      # True
s.__contains__('lo')  # use  'lo' in s
--> True
```

## Case & Stripping

```
s.upper()            # 'HELLO, WORLD!'
s.lower()            # 'hello, world!'
s.title()            # 'Hello, World!'
s.strip()            # removes whitespace
s.strip('!')         # 'Hello, World'
s.lstrip(); s.rstrip()  # one side only
```

## Splitting & Joining

```
'a,b,c'.split(',')     # ['a','b','c']
'a,b,c'.split(',', 1)  # ['a','b,c']
'hi\nthere'.splitlines() # ['hi','there']
','.join(['a','b','c'])  # 'a,b,c'
```

## Replacing & Formatting

```
s.replace('World', 'Python')
s.center(20, '-')   # '--Hello, World!--'
s.zfill(5)          # '00042' (pad zeros)
f"{3.14:.2f}"       # '3.14'
f"{'hi':>10}"       # '        hi'
```

## Checking type

```
'abc'.isalpha()     # True
'123'.isdigit()     # True
'  '.isspace()      # True
'Abc'.istitle()     # True
```

# List Methods

## Adding elements

```
lst = [3, 1, 2]
lst.append(4)          # [3,1,2,4]
lst.insert(0, 99)      # [99,3,1,2,4]
lst.extend([5, 6])     # appends iterable
lst += [5, 6]          # same as extend
```

## Removing elements

```
lst.remove(3)      # removes first 3
lst.pop()          # removes & returns last
lst.pop(0)         # removes & returns idx 0
lst.clear()        # empties the list
del lst[1:3]       # slice delete
```

## Gotcha: `remove` vs `pop`

```
# remove: by VALUE, raises if missing
# pop:    by INDEX, returns the element
[1,2,2].remove(2)  # [1,2] -- first only
```

## Searching & Counting

```
lst = [3, 1, 2, 1]
lst.index(1)      # 1  (first occurrence)
lst.count(1)      # 2
1 in lst          # True  (prefer this)
```

## Ordering

```
lst.sort()                # in-place, None
lst.sort(reverse=True)
lst.sort(key=abs)         # custom key
sorted(lst)
# new list, original unchanged
lst.reverse()             # in-place
```

## Copying

```
lst.copy()     # shallow copy
lst[:]         # also shallow
# For nested data: copy.deepcopy(lst)
```

## Other

```
[0] * 5              # [0,0,0,0,0]
list(zip(a, b))      # pair elements
list(enumerate(a))   # (idx, val) pairs
```

# Tuple Methods

## Tuples have only 2 methods:

```
t = (1, 2, 2, 3)
t.count(2)      # 2
t.index(3)      # 3  (first occurrence)
```

## But tuples support everything else:

```
len(t);  t[1];  t[1:3]
2 in t              # True
t1 + t2             # concatenate
t * 3               # repeat
sorted(t)           # returns a list
min(t); max(t); sum(t)
```

**Tip**

Tuples are immutable but can hold mutable objects. `(lst,)[0].append(1)` works fine.

## Tuple packing / unpacking:

```
a, b, *rest = (1, 2, 3, 4, 5)
# a=1, b=2, rest=[3,4,5]
first, *mid, last = range(6)
# first=0, last=5
```

# Set Methods

## Adding & Removing

```
s = {1, 2, 3}
s.add(4)            # {1,2,3,4}
s.discard(99)       # silent if missing
s.remove(99)        # KeyError if missing
s.pop()             # removes arbitrary element
s.clear()
```

## Set Operations (operators or methods)

```
a = {1,2,3};  b = {2,3,4}
a | b    # union          {1,2,3,4}
a & b    # intersection   {2,3}
a - b    # difference     {1}
a ^ b    # symmetric diff {1,4}
a.union(b)          # same as a | b
a.issubset(b)       # a <= b
a.issuperset(b)     # a >= b
a.isdisjoint(b)     # no common elements
```

## In-place variants

```
a |= b     # a.update(b)
a &= b     # a.intersection_update(b)
a -= b     # a.difference_update(b)
a ^= b     # a.symmetric_difference_update(b)
```

## Frozen sets (hashable, usable as dict key):

```
fs = frozenset([1, 2, 3])
d = {fs: 'value'}    # works!
fs.add(4)            # AttributeError
```

## Common pattern — deduplication:

```
lst = [1,2,2,3,3,3]
unique = list(set(lst))    # order lost!
# preserve order (Python 3.7+):
unique = list(dict.fromkeys(lst))
```

**△ Gotcha**

Sets require hashable elements. Lists/dicts cannot be set members — use tuples instead.

# Dictionary Methods

## Access & Safe Access

```
d = {'a': 1, 'b': 2}
d['a']              # 1, KeyError if missing
d.get('z')          # None (safe)
d.get('z', 0)       # 0   (default)
d.setdefault('c', []).append(3)
# sets d['c']=[] if missing, then appends
```

## Iteration views

```
d.keys()      # dict_keys  -- live view
d.values()    # dict_values
d.items()     # dict_items -- (k,v) pairs

for k, v in d.items():
    print(k, v)
```

## Removing

```
d.pop('a')          # returns value
d.pop('z', None)    # safe pop
d.popitem()         # LIFO (k,v) pair
d.clear()
del d['b']
```

## Merging (Python 3.9+)

```
merged = d1 | d2        # new dict
d1 |= d2                # in-place
d1.update(d2)           # also in-place
```

## Building dicts

```
dict(a=1, b=2)
dict([('a',1),('b',2)])
dict.fromkeys(['a','b'], 0)  # {'a':0,'b':0}
{k: v for k,v in items}      # comprehension
```

## collections.defaultdict:

```
from collections import defaultdict
dd = defaultdict(list)
dd['key'].append(1)  # no KeyError ever
dd['key'].append(2)
# {'key': [1, 2]}

counts = defaultdict(int)
for c in 'hello': counts[c] += 1
```

**Tip**

Dict order is insertion order (Python 3.7+).
d.copy() is a *shallow* copy

# Mutable Default Arguments

```python
def add_item(item, lst=[]):
    lst.append(item)
    return lst

print(add_item(1))
print(add_item(2))
print(add_item.__defaults__)
```

# Mutable Default Arguments — The Bug

```python
def add_item(item, lst=[]):     # list created ONCE, at definition time
    lst.append(item)
    return lst

print(add_item(1))              # [1]
print(add_item(2))              # [1, 2]   NOT [2]!
print(add_item.__defaults__)    # ([1, 2],)  the dirty default
```

The list lives on the *function object*, shared by every call that doesn't pass `lst`.

**△ Gotcha**

Same trap with `dict`, `set`, or any mutable default. Default values are evaluated **once**, at definition time.

# Mutable Default Arguments — The Fix

**Use `None` as a sentinel — immutable, idiomatic.**

```python
def add_item(item, lst=None):
    if lst is None:        # use 'is', not '=='
        lst = []           # brand-new list on every call
    lst.append(item)
    return lst

print(add_item(1))         # [1]
print(add_item(2))         # [2]  correct!
print(add_item(3, [10]))   # [10, 3]  explicit list works too
```

**Tip**

None is a singleton. `is None` is correct and avoids calling `__eq__`.

# Scoping: LEGB and `nonlocal`

Python resolves names in order: **L**ocal → **E**nclosing → **G**lobal → **B**uilt-in.

```python
x = 'global'

def outer():
    x = 'enclosing'
    def inner():
        nonlocal x        # targets the enclosing x, not the global
        x = 'changed'
    inner()
    print(x)              # 'changed'

outer()
print(x)                  # 'global'  -- untouched
```

**△ Gotcha**

Without `nonlocal`, assigning `x` in `inner` creates a brand-new *local* — the enclosing `x` is untouched. `global` → module level; `nonlocal` → nearest enclosing scope that owns the name.

# Late Binding Closures

```python
funcs = [lambda: i for i in range(5)]
print(funcs[0]())
print(funcs[3]())
```

# Late Binding Closures

```python
funcs = [lambda: i for i in range(5)]
print(funcs[0]())    # 4   -- NOT 0
print(funcs[3]())    # 4   -- NOT 3
```

The lambda captures `i` **by reference**. By call time the loop has finished: `i = 4`.

**Fix: capture at definition time with a default argument.**

```python
funcs = [lambda _=i: _ for i in range(5)]
print(funcs[0]())    # 0   ✓
print(funcs[3]())    # 3   ✓
```

# Question: Late Binding

**Question**

Predict every output. Explain why good and bad differ.

```python
def make_adders(n):
    return [lambda x, i=i: x + i for i in range(n)]

def make_adders_bad(n):
    return [lambda x: x + i for i in range(n)]

good = make_adders(3);  bad = make_adders_bad(3)

print(good[0](10))    # (A)
print(good[1](10))    # (B)
print(bad[0](10))     # (C)
print(bad[2](10))     # (D)
```

# Answer: Late Binding

**(A)** `good[0](10)`

**10.** `i=0` was bound as a default arg at creation. `10+0=10`.

**(B)** `good[1](10)`

**11.** `i=1` bound at creation. `10+1=11`.

**(C)** `bad[0](10)`

**12.** NOT 10! `i` is looked up at call time. Loop ended with `i=2`. `10+2=12`.

**(D)** `bad[2](10)`

**12.** Same reason. All three bad lambdas share the same `i=2` after the loop.

**Tip**

`i=i`: right-hand `i` is evaluated *now* and frozen as a default. Never looked up again.

# is vs == and CPython Interning

```python
a = [1,2,3];  b = [1,2,3];  c = a
print(a == b)     # True  -- same values (__eq__)
print(a is b)     # False -- different objects in memory
print(a is c)     # True  -- c is another name for a
```

**CPython caches integers** `-5..256` **and interns short strings:**

```python
x=256; y=256; print(x is y)   # True  (cached)
x=257; y=257; print(x is y)   # False (different objects!)
```

`bool` **is a subclass of** `int`**:**

```python
print(True + True)            # 2
print(isinstance(True, int))  # True
print(sum([True, False, True, True]))  # 3  -- counts True as 1
```

**△ Gotcha**

Use `is` *only* for `None`, `True`, `False`. For values, always use `==`.

# Question: is vs == with __eq__

**Question**

Predict every output. Justify each answer using internals.

```python
class Foo:
    def __eq__(self, other): return True

f1, f2 = Foo(), Foo()
print(f1 == f2)            # (1)
print(f1 is f2)            # (2)

import sys
a = sys.intern('hello world')
b = sys.intern('hello world')
print(a is b)              # (3)

lst = [1, 2, 3]
print(lst == lst[:])       # (4)
print(lst is lst[:])       # (5)
```

# Answer: `is` vs `==`

**(1) `f1 == f2`**

**True.** `==` calls `__eq__`, which unconditionally returns `True`.

**(2) `f1 is f2`**

**False.** Two separate `Foo()` calls made two objects. `id(f1) != id(f2)`.

**(3) `a is b`**

**True.** `sys.intern` forces the runtime to reuse the same string object for identical content.

**(4) `lst == lst[:]`**

**True.** Slice gives a new list with equal values. `==` compares element-by-element.

**(5) `lst is lst[:]`**

**False.** Slicing *always* returns a new list object. Same values, different address.

**Tip**

After `del x`, a new object *may* reuse the same `id()`. Never cache `id()` values.

# Shallow vs Deep Copy

| | **Assignment** | **Shallow** `copy.copy()` | **Deep** `copy.deepcopy()` |
|---|---|---|---|
| Outer object | same | new | new |
| Inner objects | same | **shared** | cloned recursively |
| Mutate inner | both see | **both see** | independent |

```python
import copy
orig = [[1, 2], [3, 4]]
s = copy.copy(orig);  d = copy.deepcopy(orig)

orig[0].append(99)
print(s[0])     # [1, 2, 99]  -- inner list shared!
print(d[0])     # [1, 2]      -- fully independent
```

**△ Gotcha**

`list[:]`, `list(x)`, `dict.copy()` are all *shallow*. Any nesting at all? Use `deepcopy`.

# Question: Nested Mutation

**Question**

What does each print produce? Which mutations are visible through b?

```python
import copy
a = {'x': [1, 2, 3], 'y': {'z': 99}}
b = copy.copy(a)
c = copy.deepcopy(a)

a['x'].append(4)      # mutate inner list
a['y']['z'] = 0       # mutate inner dict
a['w'] = 'new'        # add key to outer dict

print(b['x'])         # (1)
print(b['y']['z'])    # (2)
print('w' in b)       # (3)
print(c['x'])         # (4)
print(c['y']['z'])    # (5)
```

# Answer: Nested Mutation

**(1) `b['x']`**

`[1,2,3,4]`. `b['x']` and `a['x']` are the *same* list object. Append is visible through both.

**(2) `b['y']['z']`**

**0.** `b['y']` is the same inner dict.

**(3) `'w' in b`**

**False.** New key added to `a`'s outer dict only. `b` has its own outer dict — new keys in `a` are invisible in `b`.

**(4) `c['x']`**

`[1,2,3]`. Deep copy cloned the inner list. Mutations to `a['x']` have no effect.

**(5) `c['y']['z']`**

**99.** Deep copy also cloned the nested dict.

**Tip**

Shallow: new outer box, same inner contents.
Deep: every box redrawn at every level.

# Generator Exhaustion

```python
gen = (x**2 for x in range(5))
print(list(gen))    # [0, 1, 4, 9, 16]
print(list(gen))    # []  -- exhausted, no rewind

gen2  = (x for x in range(5))
total = sum(gen2)           # sum() fully consumes it
print(list(gen2))   # []  -- nothing left
```

**Body executes only when `next()` is called:**

```python
def gen_fn():
    print("start"); yield 1
    print("mid");   yield 2
    print("end")

g = gen_fn()    # nothing printed -- body not started yet
next(g)         # prints "start", pauses at yield 1
next(g)         # prints "mid",   pauses at yield 2
next(g)         # prints "end",   raises StopIteration
```

# MRO — C3 Linearisation

Python uses **C3**: left-to-right, depth-first, no class before all its subclasses.

```
class A: pass;  class B(A): pass
class C(A): pass;  class D(B, C): pass
print(D.__mro__)   # (D, B, C, A, object)
```

**What does `D().hi()` print?**

```
class Base:
    def hi(self): print("Base")
class A(Base):
    def hi(self): print("A"); super().hi()
class B(A):
    def hi(self): print("B"); super().hi()
class C(A):
    def hi(self): print("C"); super().hi()
class D(B, C):
    def hi(self): print("D"); super().hi()
```

# Answer: MRO + `super()` Chain

**MRO of D:**

$$D \to B \to C \to A \to \texttt{Base}$$

```
D
B
C
A
Base
```

`super()` means *next in the calling instance's MRO*, not "my direct parent".

1. `D.hi()`: prints `D`, `super()` → `B`.
2. `B.hi()`: prints `B`, `super()` → `C`.
3. `C.hi()`: prints `C`, `super()` → `A`.
4. `A.hi()`: prints `A`, `super()` → `Base`.
5. `Base.hi()`: prints `Base`. Chain ends.

**Tip**

Cooperative MI: each class calls `super()` and the MRO guarantees every class runs exactly once.

# Question: Making Mutable Objects Hashable

**Question**

What goes wrong at line 15? Fix the class using deepcopy. Then explain the __hash__/__eq__ contract.

```
1  class BadFrozenList:
2      def __init__(self, lst):
3          self._data = lst          # stores the reference
4
5      def __hash__(self):
6          return hash(tuple(self._data))
7
8      def __eq__(self, other):
9          return self._data == other._data
10
11 original = [1, 2, 3]
12 f = BadFrozenList(original)
13 d = {f: 'hello'}
14 original.append(4)        # mutate!
15 print(d[f])               # line 15 -- what happens?
```

# Answer: Hashable + deepcopy

Line 15 raises **KeyError**. `original.append(4)` mutates `f._data` (same object), which changes `hash(f)`, so Python can no longer find `f` in `d`.

```python
1  from copy import deepcopy
2
3  class FrozenList:
4      def __init__(self, lst):
5          self._data = deepcopy(lst)      # snapshot: breaks the reference
6
7      def __hash__(self):
8          return hash(tuple(self._data))
9
10     def __eq__(self, other):
11         return isinstance(other, FrozenList) \
12                and self._data == other._data
13
14 original = [1, 2, 3]
15 f = FrozenList(original)
16 d = {f: 'hello'}
17 original.append(4)          # mutates original, not f._data
18 print(d[f])                 # 'hello' -- works correctly
```

**△ Gotcha**

Contract: objects that compare == must have the same hash, and an object's hash **must not change** while it is inside a dict or set.

# Question: Pipeline Framework (Parts 1 & 2)

## Question

**Part 1:** What does `pipe(1)` print? Explain why. **Part 2:** Fix `make_multiplier_pipeline` using *two* different approaches so `pipe(1)` returns 30.

```python
def make_pipeline(*steps):
    def pipeline(data):
        result = data
        for step in steps: result = step(result)
        return result
    return pipeline

def make_multiplier_pipeline(factors):
    steps = []
    for factor in factors:
        steps.append(lambda x: x * factor)
    return make_pipeline(*steps)

pipe = make_multiplier_pipeline([2, 3, 5])
print(pipe(1))    # ?
```

# Answer: Pipeline Parts 1 & 2

**Part 1:** prints **125** ($= 5^3$), not 30. All three lambdas capture `factor` by reference. When called, the loop has ended and `factor = 5`. Each step computes $x \times 5$: $1 \to 5 \to 25 \to 125$.

**Default argument (captures at definition time):**

```

def make_multiplier_pipeline(factors):
    return make_pipeline(*[lambda x, f=f: x*f for f in factors])
```

# NumPy Essentials

## Shape, ndim, dtype

```
a = np.arange(12).reshape(3, 4)
a.shape    # (3, 4)
a.ndim     # 2
a.dtype    # int64
```

## Views vs copies

```
b = a[0:2]          # VIEW
c = a[0:2].copy()   # COPY
b[0,0] = 99         # changes a!
c[0,0] = 99         # does NOT
# b.base is a  --> True  (is a view)
```

## Axis direction

```
# axis=0: DOWN rows
# axis=1: ACROSS columns
a.sum(axis=0)    # (4,) -- col sums
a.sum(axis=1)    # (3,) -- row sums
a.mean(axis=1, keepdims=True)
# shape (3,1) -- keeps the axis
```

## Boolean indexing = copy

```
mask = a > 5
a[mask]      # new array, not a view
a[mask] = 0 # still modifies a
```

**△ Gotcha**

Slicing → view; boolean/fancy indexing → copy. Check: `b.base is a`.

# Broadcasting — The Three Rules

NumPy operates on different shapes without copying, by stretching size-1 dims.

**Rules (applied right-to-left)**

1. Different `ndim`? **Prepend 1s** to the smaller shape.
2. A size-**1** dimension stretches to match the other.
3. Sizes differ and **neither is 1** → `ValueError`.

| Shape A | Shape B | Result | Note |
|---|---|---|---|
| `(4,3)` | `(3,)` | `(4,3)` | scalar per column |
| `(3,)` | `(3,1)` | `(3,3)` | outer sum |
| `(4,1)` | `(1,3)` | `(4,3)` | outer product |
| `(2,3,4)` | `(3,4)` | `(2,3,4)` | trailing match |
| `(3,)` | `(4,)` | **Error** | $3 \neq 4$, neither 1 |

# Broadcasting Application 1: Index Grids & Masks

**Create any $m \times n$ matrix where $A_{ij} = f(i,j)$ — no loops.**

```
m, n = 4, 5
rows = np.arange(m).reshape(-1, 1)    # shape (4,1) col-vector
cols = np.arange(n).reshape(1, -1)    # shape (1,5) row-vector

# (4,1) op (1,5)  -->  broadcasts to (4,5) every time
checkerboard = (rows + cols) % 2          # 0 or 1
distance     = np.abs(rows - cols)        # D[i,j] = |i-j|
lower_tri    = (rows >= cols)             # lower-triangular mask
upper_anti   = (rows + cols < n)          # anti-diagonal mask
```

# Broadcasting Application 2: Pairwise Distances

**All $N^2$ distances between $N$ points — no Python loop.**

```
X = np.random.randn(100, 3)      # 100 points in 3-D

# X[:,None,:] has shape (100,1,3)
# X[None,:,:] has shape (1,100,3)
diff = X[:,np.newaxis,:] - X[np.newaxis,:,:]   # (100,100,3)
D    = np.sqrt(np.sum(diff**2, axis=2))        # (100,100)
```

**Faster: use $\|a-b\|^2 = \|a\|^2 + \|b\|^2 - 2a{\cdot}b$:**

```
sq = np.sum(X**2, axis=1, keepdims=True)  # (100,1)
D2 = sq + sq.T - 2 * X @ X.T              # (100,100)
D  = np.sqrt(np.clip(D2, 0, None))
```

**Tip**

`sq + sq.T` is `(N,1)+(1,N)`$\rightarrow$`(N,N)` — the outer-sum pattern from Application 1.

# Broadcasting Application 3: Numerically Stable Softmax

**Naive softmax overflows for large logits.**

```
X = np.array([[1.0, 2.0, 3.0],
              [100., 200., 300.]])     # row 2 overflows np.exp!

# WRONG:
# np.exp(X) / np.exp(X).sum(axis=1)  --> overflow + shape error

# RIGHT: subtract row-max first
X_s = X - X.max(axis=1, keepdims=True)  # (B,C)-(B,1)->(B,C)
e   = np.exp(X_s)
out = e / e.sum(axis=1, keepdims=True)  # (B,C)/(B,1)->(B,C)
```

**△ Gotcha**

Without `keepdims=True`: `.max(axis=1)` gives `(B,)`. Then `(B,C) - (B,)` aligns right: $C \neq B \rightarrow$ error (or silently wrong when $B = C$).

# Broadcasting Application 4: One-Hot & Per-Row Masks

**One-hot encoding with a single comparison:**

```
labels  = np.array([0, 2, 1, 2])        # (N,)
K       = 3
# labels[:,None] is (4,1); np.arange(K) is (3,)
one_hot = (labels[:, np.newaxis] == np.arange(K))  # (4,3)
# [[T,F,F], [F,F,T], [F,T,F], [F,F,T]]
```

**Per-row threshold — mask values below each row's median:**

```
X       = np.random.randn(5, 4)
medians = np.median(X, axis=1, keepdims=True)  # (5,1)
mask    = X > medians                          # (5,4) bool
```

**Tip**

`[:, np.newaxis]` is the standard idiom to turn a `(N,)` vector into a `(N,1)` column for row-wise operations.

# Question: Broadcasting

**Question**

(a) Shape and values of `result`? (b) Does line 9 raise an error? Why?

```
1  import numpy as np
2
3  rows = np.arange(3).reshape(-1, 1)    # (3,1)
4  cols = np.arange(4)                   # (4,)
5
6  result = rows * 10 + cols             # (a) ?
7
8  X  = np.random.randn(5, 4)
9  mu = X.mean(axis=1)                   # shape?
10 Y  = X - mu                           # (b) error?
```

# Answer: Broadcasting

**(a)** `result`

**Shape** `(3,4)`. `cols` `(4,)` prepended to `(1,4)`; both stretch to `(3,4)`. $A_{ij} = 10i + j$: `[[0,1,2,3], [10,11,12,13], [20,21,22,23]]`

**(b)** `X - mu`

**ValueError.** `mu = X.mean(axis=1)` has shape `(5,)`. `(5,4) - (5,)`: align right $\rightarrow 4 \neq 5$, neither is 1. Broadcasting fails. **Fix:** `X - mu[:, np.newaxis]` or `X.mean(axis=1, keepdims=True)`.

# Some Useful NumPy Tips

**Instant shape check in REPL:**

```
np.broadcast_shapes(
    (5,1,4), (1,3,1))
# (5,3,4) -- no arrays needed

np.broadcast_to(
    b, (4,3)).shape
# see b expanded
```

**Tip**

Run `np.broadcast_shapes` *before* the op. Raises immediately if shapes are incompatible.

# Question: Views, Axes, and Broadcasting

**Question**

Error or not? If not, give the shape or state whether X is modified.

```python
import numpy as np
X = np.arange(12).reshape(3, 4)

a  = X[:, 1]                                  # (A) shape? view?
a[0] = 99                                     # (B) changes X?

mean_rows = X.mean(axis=1)                    # (C) shape?
Y1 = X - mean_rows                            # (D) error?
Y2 = X - mean_rows[:, np.newaxis]             # (E) shape?

col_norm = X / X.sum(axis=0)                 # (F) shape?
```

# Answer: Views, Axes, and Broadcasting

**(A)**

`(3,)` — **a view.** Column slice returns a 1-D view of the data buffer.

**(B)**

**Yes, X changes.** `a` is a view; `a[0]=99` writes `X[0,1]`.

**(C)**

`(3,)`. Axis 1 collapsed; one mean per row.

**(D)**

**ValueError!** `(3,4)-(3,)`: align right $\rightarrow 4 \neq 3$.

**(E)**

`(3,4). [:,np.newaxis]` $\rightarrow$ `(3,1)`; broadcasts across columns. Each row gets its mean subtracted.

**(F)**

`(3,4). sum(axis=0)` gives `(4,)`; broadcasts to each row (column normalisation).

# Matplotlib

```python
fig, ax = plt.subplots()
ax.plot(x, y, label='A')
ax.set_title('Title')
ax.set_xlabel('x')
ax.legend()
plt.tight_layout()
plt.show()
```

**Multiple subplots:**

```python
fig, axes = plt.subplots(
    1, 2, figsize=(9, 4))
axes[0].scatter(x, y)
axes[1].bar(cats, vals)
plt.tight_layout()
```

| Plot | Call |
|---|---|
| Line | `ax.plot(x,y)` |
| Scatter | `ax.scatter(x,y)` |
| Bar | `ax.bar(l,v)` |
| Hist | `ax.hist(d,bins=20)` |
| Heatmap | `ax.imshow(M)` |
| Contour | `ax.contourf(X,Y,Z)` |

**Save:**

```python
fig.savefig('f.png',
    dpi=150,
    bbox_inches='tight')
```

# Matplotlib + NumPy: No-Loop Plotting

```python
import numpy as np, matplotlib.pyplot as plt

x = np.linspace(-3, 3, 200)
y = np.linspace(-3, 3, 200)
X, Y = np.meshgrid(x, y)              # each (200,200)
Z = np.sin(X) * np.exp(-0.3*(X**2 + Y**2))

fig, axes = plt.subplots(1, 2, figsize=(9, 3.5))

im = axes[0].imshow(Z, extent=[-3,3,-3,3],
                    origin='lower', cmap='RdBu')
fig.colorbar(im, ax=axes[0])
axes[0].set_title('Heatmap')

axes[1].contourf(X, Y, Z, levels=15, cmap='viridis')
axes[1].set_title('Contour')

plt.tight_layout()
plt.show()
```

ALL THE BEST!